

Table of Contents

1 Basic

1.1 Input

1.2 Output

2 Primitive Data Type

2.1 Assignment

2.2 Data Type Conversion

3 Operators

3.1 Numerical Operators

3.2 Assignment Operators

3.3 Comparison Operators

3.4 Bitwise Operators

3.5 Logical Operators

3.5.1 Chaining comparison

3.6 Membership operator

3.7 Object identity operator

3.8 Truth Value of an Object

4 Flow Control

4.1 Decision / Branching the Flow

4.1.1 If Statement

4.1.2 If-Else Statement

4.1.3 If-Elif-Else Statement

4.2 Loop

4.2.1 Range

4.2.2 For-In Loop

4.2.3 While Loop

4.2.4 List Comprehension

5 Collection

5.1 List

5.1.1 Slice (Colon notation)

5.2 Tuple

5.3 Dictionary

5.4 Set

6 Function

6.0.1 Hierarchical Functions

6.0.2 Recursive Function

6.0.3 Anonymous Function

6.0.4 Map function

6.0.5 Function to Evaluate a Code

7 Class and Object

7.1 Defining a Class

7.2 Inheritance

7.3 Polymorphism

8 Date and Time

This tutorial is for absolute beginners in Python. Python is a general purpose, open source programming language that is very popular. Python is also an interactive programming, that means you can directly get the results of your code.

Basic

Here is an example to use Python as a calculator

```
In [1]: 1+2+3+4+5
```

```
Out[1]: 15
```

Input

Use **input(*strMessage*)** to get input from user. The program will wait for the user to type some text or number on the keyboard and press enter.

```
In [2]: a=input("enter a number: ")
```

```
enter a number: 123
```

Output

Use **print(*list of string*)** function to displays the string value inside the parentheses on the screen. When *print()* function is used together with *input()* function, the print will be done only after the user input.

```
In [3]: print('What is your name?')
name=input()
print('Your name is ' + name)
```

```
What is your name?
Chinmaya Mahapatra
Your name is Chinmaya Mahapatra
```

Primitive Data Type

Python has several primitive data types:

1. Integer (any whole number, either positive or negative)
2. Float (any number with decimal; dot represent the separation between the whole number and the decimal part)
3. String (start with either single quote " or double quote ""
4. Boolean (True, False, or used in comparison)
5. Complex (real and imaginary)

Python also has collection of the primitive data type such as

1. List
2. Tuple
3. Dictionary
4. Set

We will discuss the collection of primitive data types in later sections.

Here are some examples of the primitive data types. Note: symbol # is the start of a comment that will not be executed by Python

```
In [4]: 1,2,3 # integers
```

```
Out[4]: (1, 2, 3)
```

```
In [5]: 1.1, 2., 3.6 # floats
```

```
Out[5]: (1.1, 2.0, 3.6)
```

```
In [6]: '1', 'abc', '3.6' # strings
```

```
Out[6]: ('1', 'abc', '3.6')
```

```
In [7]: True, False, 1==2, 3.5==3.5 # Boolean
```

```
Out[7]: (True, False, False, True)
```

```
In [8]: complex('1+2j')+complex('3+5j') # complex numbers
```

```
Out[8]: (4+7j)
```

Assignment

You can store the value of an integer, float or string into a variable as the following example

```
In [9]: a=2
        b=3+a
        c=b-a
        c
```

```
Out[9]: 3
```

Name of variables (and functions and classes) are case sensitive.

In Python you can assign multiple values at once.

```
In [10]: a,b,c=1,2,3  
        d=[a,b,c]  
        d
```

```
Out[10]: [1, 2, 3]
```

```
In [11]: a=b=c=0  
        d=[a,b,c]  
        d
```

```
Out[11]: [0, 0, 0]
```

To swap two values, we simply do the following:

```
In [12]: a=3  
        b=5  
        a,b=b,a  
        print(a,b)
```

```
5 3
```

Data Type Conversion

You can convert a value into a string using function **str()**.

You can convert a value into an integer using function **int()**.

You can convert a value into a float using function **float()**.

You can convert a value into a float using function **bool()**.

You can convert a value into a float using function **complex()**.

```
In [13]: str(3)
```

```
Out[13]: '3'
```

```
In [14]: int(3.45)
```

```
Out[14]: 3
```

```
In [15]: float('2.5')+2.5
```

```
Out[15]: 5.0
```

```
In [16]: bool(3)
```

```
Out[16]: True
```

```
In [17]: complex(3,5)
```

```
Out[17]: (3+5j)
```

Operators

Numerical Operators

Numerical operators are either used toward one or two values and produce a single value. The following are numerical operators and most useful numerical functions.

operator	Description
$x + y$	sum of x and y
$x - y$	difference of x and y
$x * y$	multiplication of x and y
x / y	division of x and y
$x \% y$	remainder of x / y (modulus)
$x // y$	floored quotient of x and y (integer division)
<code>abs(x)</code>	absolute value or magnitude of x
$-x$	x negated
$+x$	x unchanged
$x ** y$	x to the power y
<code>pow(x, y)</code>	x to the power y
<code>math.sqrt(x)</code>	square root of x
<code>math.floor(x)</code>	the greatest integer $\leq x$
<code>math.ceil(x)</code>	the least integer $\geq x$
<code>math.trunc(x)</code>	x truncated to integer
<code>round(x[, n])</code>	x rounded to n digits, rounding half to even. If n is omitted, it defaults to 0.
<code>divmod(x,y)</code>	the pair $(x // y, x \% y)$
<code>int(x)</code>	convert x to integer
<code>float(x)</code>	convert x to float
<code>complex(re,im)</code>	convert x to complex number
<code>c.conjugate()</code>	conjugate of the complex number c

```
In [18]: import math

x=-30; y=2.1
print(str(x) + "+" + str(y) + " = ",x+y)
print(str(x) + "-" + str(y) + " = ",x-y)
print(str(x) + "*" + str(y) + " = ",x*y)
print(str(x) + "/" + str(y) + " = ",x/y)
print(str(x) + "%" + str(y) + " = ",x%y)
print("abs("+ str(x) +") = ",abs(x))
print("-" + str(y) + " = ",-y)
print"+" + str(y) + " = ",+y)
print(str(x) + "**" + str(y) + " = ",x**y)
print("pow(" + str(x) + "," + str(y) + ") = ",pow(x,y))
print("math.sqrt("+str(-x) + ") = ",math.sqrt(-x)) # only for positive number
print("math.floor(math.sqrt("+str(-x) + ")) = ",math.floor(math.sqrt(-x)))
print("math.ceil(math.sqrt("+str(-x) + ")) = ",math.ceil(math.sqrt(-x)))
print("math.trunc("+str(y) + ") = ",math.trunc(y))
print("divmod("+str(x) + "," + str(y) + ") = ",divmod(x,y))
print("int("+ str(y) + ") = ",int(y))
print("float("+ str(x) + ") = ",float(x))
print("complex("+ str(x) + "," + str(y) + ") = ",complex(x,y))
print("complex("+ str(x) + "," + str(y) + "),conjugate() = ",complex(x,y).conjugate
())

-30+2.1 = -27.9
-30-2.1 = -32.1
-30*2.1 = -63.0
-30/2.1 = -14.285714285714285
-30%2.1 = 1.5000000000000013
abs(-30) = 30
-2.1 = -2.1
+2.1 = 2.1
-30**2.1 = (1202.7101066340504+390.78420250378167j)
pow(-30,2.1) = (1202.7101066340504+390.78420250378167j)
math.sqrt(30) = 5.477225575051661
math.floor(math.sqrt(30)) = 5
math.ceil(math.sqrt(30)) = 6
math.trunc(2.1) = 2
divmod(-30,2.1) = (-15.0, 1.5000000000000013)
int(2.1) = 2
float(-30) = -30.0
complex(-30,2.1) = (-30+2.1j)
complex(-30,2.1),conjugate() = (-30-2.1j)
```

Assignment Operators

operator	Description
x += y	x = x + y
x -= y	x = x - y
x *= y	x = x * y
x /= y	x = x / y
x %= y	x = x % y

```
In [19]: x=-30; y=2.1
x+=y
print("x = ",x,"; y = ",y)

x = -27.9 ; y = 2.1
```

```
In [20]: x=-30; y=2.1
        x-=y
        print("x = ",x,"; y = ",y)

x =  -32.1 ; y =  2.1
```

```
In [21]: x=-30; y=2.1
        x*=y
        print("x = ",x,"; y = ",y)

x =  -63.0 ; y =  2.1
```

```
In [22]: x=-30; y=2.1
        x/=y
        print("x = ",x,"; y = ",y)

x =  -14.285714285714285 ; y =  2.1
```

```
In [23]: x=-30; y=2.1
        x%=y
        print("x = ",x,"; y = ",y)

x =  1.5000000000000013 ; y =  2.1
```

Comparison Operators

You can use comparison operators to compare two values and return to a single Boolean value. The following are comparison operators

==	Equal to
!=	Not equal to
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to

An integer is treated equal as float if they have the same value. String are not treated equal to number. You cannot compare number and string. See the following examples.

```
In [24]: 23==23.0 # the same value, thus treat integer is equal to float
```

Out[24]: True

```
In [25]: 23==23.5 # not the same value
```

Out[25]: False

```
In [26]: 23=='23' # number is not equal to string
```

Out[26]: False

```
In [27]: 23<=23.5 # you can compare integer and float
```

Out[27]: True

Bitwise Operators

Bitwise operation applied to positive integers. The integers are converted to binary, then apply the operator and converted back to decimal. Three main bitwise operators are

Operator	Description
x & y	bitwise and
x y	bitwise or
x ^ y	bitwise xor
~ x	bitwise inversion of x which is equal to -(x+1)
x >> y	bitwise shift right will returns x with the bits shifted to the right by y places
x << y	bitwise shift left will returns x with the bits shifted to the left by y places (and new bits on the right-hand-side are zeros). This is the same as multiplying x by 2**y.

Example:

Decimal	Binary	Description
2	010	x = first Operant
5	101	y = second Operant
0	000	x & y = result

Decimal	Binary	Description
2	010	x = first Operant
5	101	y = second Operant
7	111	x y = result

Decimal	Binary	Description
2	010	x = first Operant
5	101	y = second Operant
7	111	x ^ y = result

Distinguish bitwise operation from Logical Operation and Set operation

```
In [28]: x, y =2,5
print("bin(" +str(x)+")="+bin(x),"; bin(" + str(y) + ") = "+bin(y))
print(str(x) + "&" + str(y) + " = ", x&y)
print(str(x) + "|" + str(y) + " = ", x|y)
print(str(x) + "^" + str(y) + " = ", x^y)
print("~" + str(x) + " = ", ~x)
print(str(x) + ">>" + str(y) + " = ", x>>y)
print(str(x) + "<<" + str(y) + " = ", x<<y)
```

```
bin(2)=0b10 ; bin(5) = 0b101
2&5 = 0
2|5 = 7
2^5 = 7
~2 = -3
2>>5 = 0
2<<5 = 64
```


Notice in the example above that the bitwise *not* produce negative value because python applies it to signed integer. If you want to use for image processing, the following bitwise *not* is useful.

```
In [29]: def bitwise_not(n, numbits=8):  
         return (1 << numbits) - 1 - n  
  
         bitwise_not(2)
```

Out[29]: 253

Logical Operators

Three main logic operators are **and**, **or**, **not**.

```
In [30]: not True
```

Out[30]: False

```
In [31]: True and False
```

Out[31]: False

```
In [32]: True or False
```

Out[32]: True

When logical operator is applied into number, a non-zero is considered as True. Zero is considered as False. Logical *not* of a non-zero would be False. Not 0 would be True. If both operands are non-zero:

- logical *and* will return the second operand
- logical *or* will return the first operand

```
In [33]: x, y = 1,-2  
         print(str(x) + " and " + str(y) + " = ", x and y)  
         print(str(x) + " or " + str(y) + " = ", x or y)  
         print("not " + str(y) + " = ",not y)
```

```
1 and -2 = -2  
1 or -2 = 1  
not -2 = False
```

Chaining comparison

Comparison can be chained, but the evaluation is every pair. As if there is **and** between the two.

```
In [34]: a=5  
         2<a<6 # equal to (2<5) and (5<6)
```

Out[34]: True

```
In [35]: 0<=a<2 # equal to (0<=a) and (a<2)
```

Out[35]: False

Membership operator

Operator **in** return True if it find a variable in a specified sequence.

```
In [36]: 'good' in 'Apple has a good taste'
```

```
Out[36]: True
```

```
In [37]: 'bad' not in 'Durian has bad odor'
```

```
Out[37]: False
```

Object identity operator

Operator **is** return True if the variables on either side of the operator point to the same object.

```
In [38]: a=234  
         b=a  
         a is b # return True because both variables a and b are pointing to the same object
```

```
Out[38]: True
```

```
In [39]: a=123.5  
         b=123.5  
         a is b # rturn False even if the same value because not the same object
```

```
Out[39]: False
```

Watch out: **is** operator will return True if they have the same value. This behavior is only correct for integer and string

```
In [40]: a='123'  
         b='123'  
         a is b # is operator will return true if they have the same value only for integer and string
```

```
Out[40]: True
```

```
In [41]: a=123  
         b=123  
         a is b
```

```
Out[41]: True
```

Truth Value of an Object

Every object has a boolean value. The following elements are False:

- None
- False
- 0 (from any type of integer, float to complex such as 0, 0.0, 0j)
- Empty sequence or collections or mapping: "", (), [], {}
- Objects from classes that have the special method **nonzero**
- Objects from classes that implements **len** to return False or zero

All other values are considered True

Flow Control

Decision / Branching the Flow

In Python, you can use the following conditional flow control:

- if statement
- if-else statement
- if-elif-else statement

If Statement

```
In [42]: a=23
         if a==23:
             print('it is my number!')

it is my number!
```

If-Else Statement

If-then statement is similar to a mathematical function $f = \begin{cases} \text{'Good morning!'} & \text{if } a = \text{'hello'} \\ \text{'Good afternoon!'} & \text{if } a \neq \text{'hello'} \end{cases}$

```
In [43]: a='helloo'
         if a=='hello':
             print('Good morning!')
         else:
             print('Good afternoon!')

Good afternoon!
```

If-Elif-Else Statement

if-elif-else statement is similar to a mathematical function $f = \begin{cases} \text{'negative number'} & \text{if } x < 0 \\ \text{'zero'} & \text{if } x = 0 \\ \text{'positive number'} & \text{if } x > 0 \end{cases}$

```
In [44]: x=-231.12
print(x)
if float(x)<0:
    print('negative number')
elif float(x)==0:
    print('zero')
else:
    print('positive number')
```

```
-231.12
negative number
```

Loop

The following flow control are used to repeat or to iterate in Python:

- range
- for
- while

the following statements are useful to break or continue the loop:

- break (to stop the execution of the loop)
- continue (to continue with the next iteration of the loop)

Range

To show the values in a range, use **list(range(start [, end][, step]))**. The *end* is exclusive while the *start* is inclusive.

```
In [45]: list(range(1,10,2))
```

```
Out[45]: [1, 3, 5, 7, 9]
```

```
In [46]: list(range(2,5))
```

```
Out[46]: [2, 3, 4]
```

```
In [47]: list(range(5))
```

```
Out[47]: [0, 1, 2, 3, 4]
```

For-In Loop

```
In [48]: fruits = ['apple', 'banana', 'cherry', 'durian']
for f in fruits:
    print(f, len(f))  # built-in function len is to count the number of characters in
a word
```

```
apple 5
banana 6
cherry 6
durian 6
```

```
In [49]: for k in range(5):  
        print(k)
```

```
0  
1  
2  
3  
4
```

```
In [50]: for num in range(1, 10):  
        if num % 2 == 0:  
            print(num, "is an even number")  
            continue  
        print(num, "is an odd number")
```

```
1 is an odd number  
2 is an even number  
3 is an odd number  
4 is an even number  
5 is an odd number  
6 is an even number  
7 is an odd number  
8 is an even number  
9 is an odd number
```

A loop statements may have an **else** clause; it is executed when the loop terminates through exhaustion of the list (with **for**) or when the condition becomes False (with **while**), but not when the loop is terminated by a **break** statement.

```
In [51]: for n in range(2, 10):  
        for x in range(2, n):  
            if n % x == 0:  
                print(n, 'equals', x, '*', n//x)  
                break  
        else:  
            print(n, 'is a prime number')
```

```
2 is a prime number  
3 is a prime number  
4 equals 2 * 2  
5 is a prime number  
6 equals 2 * 3  
7 is a prime number  
8 equals 2 * 4  
9 equals 3 * 3
```

While Loop

```
In [52]: a=0  
while a<5:  
    print(a)  
    a=a+1
```

```
0  
1  
2  
3  
4
```

```
In [53]: a=0
while True:
    if a>=5:
        break
    else:
        a+=1
        print(a)
```

```
1
2
3
4
5
```

List Comprehension

In Python, you can construct a list by writing the criteria based on sequence S.

Syntax:

- `[f(x) for x in S]`
- `[x for x in S if f(x)]`

```
In [54]: square=[x**2 for x in range(10)] # return list of square integers
square
```

```
Out[54]: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```
In [55]: [x for x in square if x%2==0] # return list of even square integers
```

```
Out[55]: [0, 4, 16, 36, 64]
```

Collection

We will discuss three type of collection of the primitive data types:

1. List
2. Tuple
3. Dictionary
4. Set

List

A list is comma-separated values (items) between square brackets []. Think of a list as an array of ordered sequence. The index of the array start from 0. List is a *mutable* data type. You can add value, remove and change the items in a list.

```
In [56]: x=[1,2,3]
y=[4,5,6]
x+y
```

```
Out[56]: [1, 2, 3, 4, 5, 6]
```

```
In [57]: for x in [4,5,6]: print(x)
```

```
4
5
6
```

```
In [58]: x=['Hello ']  
x*4
```

```
Out[58]: ['Hello ', 'Hello ', 'Hello ', 'Hello ']
```

```
In [59]: m = "Peter"  
a=[1,2,3,'Paul',2.35,m]  
a
```

```
Out[59]: [1, 2, 3, 'Paul', 2.35, 'Peter']
```

We can change assign values to the element of the list:

```
In [60]: a[0]='Peter'  
a
```

```
Out[60]: ['Peter', 2, 3, 'Paul', 2.35, 'Peter']
```

To add at the back of a list, use **append()** method.

```
In [61]: a.append('last')  
a
```

```
Out[61]: ['Peter', 2, 3, 'Paul', 2.35, 'Peter', 'last']
```

Use **len()** function to check the total number of elements in a list.

```
In [62]: len(a)
```

```
Out[62]: 7
```

We can remove the element of a list using **del**

```
In [63]: del(a[1:3])  
a
```

```
Out[63]: ['Peter', 'Paul', 2.35, 'Peter', 'last']
```

We can also remove an element using **remove()** method

```
In [64]: a.remove(2.35)  
a
```

```
Out[64]: ['Peter', 'Paul', 'Peter', 'last']
```

We can use **insert(position, object)** method to put an object at some index position.

```
In [65]: a.insert(4, 'last')
a
```

```
Out[65]: ['Peter', 'Paul', 'Peter', 'last', 'last']
```

We can use **sort()** method to sort a list. Sorting only happens if the data type are the same for all items in the list.

```
In [66]: a.sort()
a
```

```
Out[66]: ['Paul', 'Peter', 'Peter', 'last', 'last']
```

Another way to sort a list is to use Python built-in function **sorted()**.

```
In [67]: sorted(a)
```

```
Out[67]: ['Paul', 'Peter', 'Peter', 'last', 'last']
```

```
In [68]: a.reverse()
a
```

```
Out[68]: ['last', 'last', 'Peter', 'Peter', 'Paul']
```

we can also use **count()** method to count how many times the objects happen in a list

```
In [69]: a.count('Peter')
```

```
Out[69]: 2
```

A list can be nested and the number of element is not necessarily the same.

```
In [70]: x=[['a'], ['b', 'c'], ['d', 'e', 'f']]
x
```

```
Out[70]: [['a'], ['b', 'c'], ['d', 'e', 'f']]
```

A list can be generated over a function

```
In [71]: y=[1,2,3,4,5]
def f(x):
    return x**2

[f(x) for x in y]
```

```
Out[71]: [1, 4, 9, 16, 25]
```

A list can be constructed using **for** loop.

```
In [72]: [y+1 for y in range(5)]
```

```
Out[72]: [1, 2, 3, 4, 5]
```



```
In [73]: x=[1,3,5,9]
        [2 + y*2 for y in x if y>=3]
```

```
Out[73]: [8, 12, 20]
```

```
In [74]: x=[1,2,3]
        [[a,b] for a in x for b in x]
```

```
Out[74]: [[1, 1], [1, 2], [1, 3], [2, 1], [2, 2], [2, 3], [3, 1], [3, 2], [3, 3]]
```

The **zip()** function turns n lists (list1, list2, ...) into one list of n-tuples, where each n-tuple (a,b,...) has its first element (a) from the first list (list1), the second element (b) from the second list (list2), and so forth.

```
In [75]: x=[1,2,3]
        y=[4,5,6]
        [[a,b] for a,b in zip(x,y)]
```

```
Out[75]: [[1, 4], [2, 5], [3, 6]]
```

To copy a list, use Python module name **copy**.

```
In [76]: import copy
        y=[100,67,56,12,99]
        x=copy.copy(y)
        x[0]=101
        print(y,x)
```

```
[100, 67, 56, 12, 99] [101, 67, 56, 12, 99]
```

To make the layout nicer and more readable, use **pprint** module that offers a pretty print functionality.

```
In [77]: import pprint
        x=[1,3,4,7]
        table=[[a,b] for a in x for b in x]
        pprint.pprint(table)
```

```
[[1, 1],
 [1, 3],
 [1, 4],
 [1, 7],
 [3, 1],
 [3, 3],
 [3, 4],
 [3, 7],
 [4, 1],
 [4, 3],
 [4, 4],
 [4, 7],
 [7, 1],
 [7, 3],
 [7, 4],
 [7, 7]]
```

Slice (Colon notation)

A list can be sliced using colon notation (*start:end:step*). Similar to **range()**, the *start* is inclusive and the *end* is exclusive. If the *start* is omitted, it starts from beginning of the list. If the *end* is omitted, it enumerate up to the end of the list. If the *step* is omitted, the *step*=1.

The index of a list start with 0. Negative index means it starts from the back.

```
In [78]: b=range(1,15)
        list(b)
```

```
Out[78]: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
```

```
In [79]: list(b[2:10:2])
```

```
Out[79]: [3, 5, 7, 9]
```

```
In [80]: list(b[-1:9:-1])
```

```
Out[80]: [14, 13, 12, 11]
```

```
In [81]: b=[3, 3.5, 4.2, 8.1]
        b[2:]
```

```
Out[81]: [4.2, 8.1]
```

```
In [82]: b[:3]
```

```
Out[82]: [3, 3.5, 4.2]
```

```
In [83]: b[1:3]
```

```
Out[83]: [3.5, 4.2]
```

```
In [84]: x=[1,3,4,7]
        table=[[a,b] for a in x for b in x]
        table[0:6]
```

```
Out[84]: [[1, 1], [1, 3], [1, 4], [1, 7], [3, 1], [3, 3]]
```

```
In [85]: table[0:6][3:]
```

```
Out[85]: [[1, 7], [3, 1], [3, 3]]
```

Tuple

A tuple is an an ordered sequence, similar to a constant list. However, a list is *mutable* data type while a tuple is *immutable* data type. Once a tuple is defined, we cannot modify, append or remove any item. Tuple is denoted by parentheses. String data type in Python behaves like tuple.

```
In [86]: x=(1,2,3)
        y=(4,5,6)
        x+y
```

```
Out[86]: (1, 2, 3, 4, 5, 6)
```

```
In [87]: for x in (4,5,6): print(x)
```

```
4
5
6
```

```
In [88]: x=('Hello ')
x*4
```

```
Out[88]: 'Hello Hello Hello Hello '
```

To convert a list or a dictionary into a tuple, use **tuple()** function.

```
In [89]: x={4,9,12}
y=tuple(x)
y
```

```
Out[89]: (9, 12, 4)
```

Dictionary

A dictionary is a key-value pair unordered set. Dictionary is denoted by curly bracket {}.

```
In [90]: x={'fruit':'apple', 'color':'red', 'taste': 'sweet'}
x
```

```
Out[90]: {'color': 'red', 'fruit': 'apple', 'taste': 'sweet'}
```

Access and alter the value of a dictionary by the key

```
In [91]: x['color']='green'
```

```
In [92]: 'the ' + x['color'] + ' ' + x['fruit'] + ' is ' + x['taste']
```

```
Out[92]: 'the green apple is sweet'
```

Use **pop()** method to return the value, given the key.

```
In [93]: x={'fruit':'apple', 'color':'red', 'taste': 'sweet', 'shape':'round'}
x.pop('fruit')
```

```
Out[93]: 'apple'
```

To change the key, we can utilize the **pop** method: *mydict[new_key] = mydict.pop(old_key)*

```
In [94]: x={'fruit':'apple', 'color':'red', 'taste': 'sweet'}
x['colour']=x.pop('color')
x
```

```
Out[94]: {'colour': 'red', 'fruit': 'apple', 'taste': 'sweet'}
```

The key in a dictionary must be unique, otherwise the later key will replace earlier key

```
In [95]: x={'fruit':'apple', 'fruit':'banana', 'taste': 'sweet'}
x
```

```
Out[95]: {'fruit': 'banana', 'taste': 'sweet'}
```

The value can be repeated, as long as it is for different key. Order does not matter.

```
In [96]: x={'fruit':'apple', 'color':'sweet', 'taste': 'sweet'}
x
```

```
Out[96]: {'color': 'sweet', 'fruit': 'apple', 'taste': 'sweet'}
```

The **keys()** method will return tuple of keys in a dictionary. Remember tuple is immutable.

```
In [97]: x={'fruit':'apple', 'color':'red', 'taste': 'sweet'}
for k in x.keys():
    print(k)
```

```
fruit
taste
color
```

The **values()** method will return tuple of values in a dictionary.

```
In [98]: for v in x.values():
        print(v)
```

```
apple
sweet
red
```

The **items()** method will return tuple of key and value in a dictionary.

```
In [99]: for i in x.items():
        print(i)
```

```
('fruit', 'apple')
('taste', 'sweet')
('color', 'red')
```

We can separate the key and the value from item.

```
In [100]: for k,v in x.items():
        print(k,': ', v)
```

```
fruit :  apple
taste :  sweet
color :  red
```

Use **in** and **not in** operators to check whether a certain key or value exist in a dictionary.

```
In [101]: 'apple' in x.values()
```

```
Out[101]: True
```

```
In [102]: 'color' not in x.keys()
```

```
Out[102]: False
```

To remove one item in dictionary, use **del**.

```
In [103]: x={'fruit':'apple', 'color':'red', 'taste': 'sweet', 'shape':'round'}
del x['taste']
x
```

```
Out[103]: {'color': 'red', 'fruit': 'apple', 'shape': 'round'}
```

To remove all items in a dictionary, use **clear()** method.

```
In [104]: x={'fruit':'apple', 'color':'red', 'taste': 'sweet', 'shape':'round'}
x.clear()
x
```

```
Out[104]: {}
```

To delete the dictionary variable, use **del**.

```
In [105]: x={'fruit':'apple', 'color':'red', 'taste': 'sweet', 'shape':'round'}
del x
```

To convert from key-value pair of tuple sequence into a dictionary use **dict()** function.

```
In [106]: y=[('fruit','apple'), ('color','red'),('taste', 'sweet')]
x=dict(y)
x
```

```
Out[106]: {'color': 'red', 'fruit': 'apple', 'taste': 'sweet'}
```

To create a dictionary from two arrays of keys and values, use **zip()** function.

```
In [107]: keys=['fruit', 'color', 'taste', 'shape']
values=['apple','red', 'sweet','round']
x=list(zip(keys,values))
print(x)
y=dict(x)
y

[('fruit', 'apple'), ('color', 'red'), ('taste', 'sweet'), ('shape', 'round')]
```

```
Out[107]: {'color': 'red', 'fruit': 'apple', 'shape': 'round', 'taste': 'sweet'}
```

To build dictionary from a range, use iterator

```
In [108]: {y: y%3 for y in range(1,15,2)}
```

```
Out[108]: {1: 1, 3: 0, 5: 2, 7: 1, 9: 0, 11: 2, 13: 1}
```

In general, a dictionary is unordered sequence, thus it cannot be sorted. To sort a dictionary by key or by value, you need to use **OrderedDict**.

```
In [109]: import collections
y={'color': 'red', 'fruit': 'apple', 'shape': 'round', 'taste': 'sweet'}
od = collections.OrderedDict(sorted(y.items()))
od # sorted by key
```

```
Out[109]: OrderedDict([('color', 'red'),
                        ('fruit', 'apple'),
                        ('shape', 'round'),
                        ('taste', 'sweet')])
```

```
In [110]: od = collections.OrderedDict(sorted(y.items(), key=lambda k: k[1], reverse=True))
od # sorted by value
```

```
Out[110]: OrderedDict([('taste', 'sweet'),
                        ('shape', 'round'),
                        ('color', 'red'),
                        ('fruit', 'apple')])
```

Using function **sorted()** will return a sorted list of the keys

```
In [111]: y={'color': 'red', 'fruit': 'apple', 'shape': 'round', 'taste': 'sweet'}
sorted(y)
```

```
Out[111]: ['color', 'fruit', 'shape', 'taste']
```

To sort the values and return a list, we can use function **sorted()** with argument of the dictionary values.

```
In [112]: sorted(y.values())
```

```
Out[112]: ['apple', 'red', 'round', 'sweet']
```

Set

A set is an unordered collection with no duplicate items. A set is a dictionary without key. Set operations are

- union: |
- intersection: &
- difference: -
- symmetric difference: ^

```
In [113]: y={3, 4, 8, 12, 8, 2, 3, 4, 12, 3, 4}
y # set gives unique values
```

```
Out[113]: {2, 3, 4, 8, 12}
```

```
In [114]: x={'apple', 'banana', 'cherry', 'durian', 'kiwi'}
y={'durian', 'kiwi', 'mango', 'orange', 'pineapple'}
x|y # set union
```

```
Out[114]: {'apple', 'banana', 'cherry', 'durian', 'kiwi', 'mango', 'orange', 'pineapple'}
```

```
In [115]: x&y # set intersection
```

```
Out[115]: {'durian', 'kiwi'}
```

```
In [116]: x-y # set difference
```

```
Out[116]: {'apple', 'banana', 'cherry'}
```

```
In [117]: x^y # set symmetric difference
```

```
Out[117]: {'apple', 'banana', 'cherry', 'mango', 'orange', 'pineapple'}
```

```
In [118]: y-x # set difference
```

```
Out[118]: {'mango', 'orange', 'pineapple'}
```

```
In [119]: x={2}
y={5}
print(str(x) + "&" + str(y) + "=", x&y) # set intersection
print(str(x) + "|" + str(y) + "=", x|y) # set union
print(str(x) + "^" + str(y) + "=", x^y) # set symmetric difference

{2}&{5}= set()
{2}|{5}= {2, 5}
{2}^5= {2, 5}
```

Distinguish set operation from bitwise operation

```
In [120]: x, y = 2, 5
print(str(x) + "&" + str(y) + "=", x&y) # bitwise and
print(str(x) + "|" + str(y) + "=", x|y) # bitwise or
print(str(x) + "^" + str(y) + "=", x^y) # bitwise xor
```

```
2&5= 0
2|5= 7
2^5= 7
```

Function

Function is a mini program that consists of several statements and call them as single function name. Function can receive no argument or any number of arguments. Function can return a value following a keyword **return**. You can think a function as a blackbox that receives inputs and produces an output.

```
In [121]: def f(a,b):
           return a+b
```

```
f(2,3)
```

```
Out[121]: 5
```

The return value is not necessarily one value. Some function receive no argument.

```
In [122]: def f():
           a=5
           b=7
           c=9
           return a,b,c

           x,y,z=f()
           w=[x,y,z]
           w
```

Out[122]: [5, 7, 9]

To make function that produces no output, simply skip the **return** statement. Implicitly, this function will return *None* object.

Documentation of the function should contain a short description of the purpose of the function and explain what the different arguments and return values are. This documentation strings are usually enclosed in triple quotes `"""` or `'''`, which allow the string to span several lines.

```
In [123]: def h():
           '''return none'''

           if h() is None:
               print("function h() was called")

           function h() was called
```

Some arguments of the function can have a default value.

```
In [124]: def k(a,b=3.2, c=True):
           if c:
               return a+b

           k(5)
```

Out[124]: 8.2

Variables outside the function has global scope. Variables defined inside a function has only local scope inside that function. In the example below, variable *a* has global scope that function *g()* can access it. Variable *b* has local scope that is available inside the function.

```
In [125]: a=5
           def g():
               b=a+2
               print('b = ',b,'; a = ',a)

           g()

           b = 7 ; a = 5
```

To defend your function against mistake in input, use **try-except** syntax


```
In [126]: def f1(a,b,c=3):
            try:
                d=a+b+c
            except (RuntimeError, TypeError, NameError, ValueError):
                d = 0
            return d

print (f1('a',2))
print (f1(2,3))

0
8
```

Hierarchical Functions

In Python, you can define functions inside a function. This will create hierarchical functions.

```
In [127]: def f_top(a):
            b="magic-" + str(a) + " "
            def f_mid(b):
                c=[b*2, b+"is happening"]
                def f_bottom(c):
                    return [c , "boom!"]
                return f_bottom(c)
            return f_mid(b)

f_top("word")
```

```
Out[127]: [['magic-word magic-word ', 'magic-word is happening'], 'boom!']
```

Recursive Function

Function can call other functions. When a function call itself, it is called recursive function. Any recursive function must have a base value.

```
In [128]: def fibonacci(n):
            if n < 2:
                return n # this is base value
            return fibonacci(n-2) + fibonacci(n-1) # this is the recursion

print([fibonacci(x) for x in range(1,20)])

[1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181]
```

Anonymous Function

In Python, a function can be assigned into a variable. A function without a name is called anonymous function. To make anonymous function, use **lambda** statement.

Syntax: *varName* = **lambda** *argList* : *expression*

```
In [129]: h = lambda x: x**2 + 1
h(3)
```

```
Out[129]: 10
```

The above anonymous function is equivalent to

```
In [130]: def h(x):  
           return x**2 + 1  
  
h(3)
```

Out[130]: 10

The following anonymous function receive several arguments

```
In [131]: affine = lambda a,x,b: a*x+b  
affine(3,4,5)
```

Out[131]: 17

Map function

function **map()** is useful to apply the function to all the elements of the sequence.

Syntax: *iterator* = **map**(*function*, *sequences*)

```
In [132]: a=[3,1,3,5]  
x=[4,2,5,1]  
b=[5,3,4,2]  
list(map(affine,a,x,b))
```

Out[132]: [17, 5, 19, 7]

```
In [133]: list(map(lambda x: x**2, range(10)))
```

Out[133]: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

Function **map** is actually the same as list comprehensions. We can also apply this technique to a list of functions instead of a list of values.

```
In [134]: from math import sin, cos, tan, pi  
  
def mapFunction(x,functions):  
    return[ func(x) for func in functions ]  
  
functions = (sin, cos, tan)  
mapFunction(pi/2,functions)
```

Out[134]: [1.0, 6.123233995736766e-17, 1.633123935319537e+16]

Function to Evaluate a Code

Use function **eval(string)** to run a string code. The simplest would be to convert string to a numerical value. However, any valid Python expression stored as text in a string can be turned into Python code and run using **eval**.

eval() is a function with delayed execution.

```
In [135]: eval('12.5+13.5')
```

```
Out[135]: 26.0
```

```
In [136]: def h(x):  
           return x**2 + 1  
  
eval('h(3)+h(5)')
```

```
Out[136]: 36
```

Class and Object

Python is object oriented programming language. You can define a class and object. Everything in python is an object of certain type. Each data type is an object of the particular class of the data type. Each function is an object of function class.

Object is a module consist of data and functions. The data inside an object is called *object property* or *attribute* while the functions inside an object are called *methods*. Any object should have a blue print or a pattern that create that object. The blue print of objects of similar type is called a *class*.

To know the class of an object, use **type()** function.

```
In [137]: a="abcd"  
          type(a)
```

```
Out[137]: str
```

Defining a Class

To define a class simply type

```
class className([optional input arguments]):
```

To define an object, we use the following syntax:

```
objectName = ClassName()
```

To access object property use the following syntax:

```
varName=objectName.propertyName
```

To change the value of an object property:

```
objectName.propertyName=varName
```

To call object method use the following syntax:

```
objectName.methodName()
```

```
In [138]: class MyFirstClass():
            b=8                                # initial class property
            def myFunc1(self, a):
                self.b=3                        # class property change inside this method
                b=5                             # this is a local variable inside this function
                print("argument=",a)

myObj=MyFirstClass()    # defining an object of a class
print(myObj.b)           # print class property
print(myObj.myFunc1(25)) # print result of class method
print(myObj.b)           # class property change because we called the class method
myObj.b=13               # we alter the value of object property
print(myObj.b)           # print the result of our change

8
argument= 25
None
3
13
```

To find out about what is available in an object, Python exposes everything that exists in an object when you use the **dir()** function.

vars() function shows the variables and the values.

```
In [139]: print(type(myObj))    # show the clas name
           print(dir(myObj))    # show all accessible properties and methods
           print(vars(myObj))   # show the attribute and its value = myObj.__dict__
           print(getattr(myObj,'b')) # show the value of the attribute

<class '__main__.MyFirstClass'>
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__', '__gt__', '__hash__', '__init__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__', 'b', 'myFunc1']
{'b': 13}
13
```

Inheritance

An object can inherit the properties and method of other objects. To set the inheritance, put the super class into argument of the class. Python allows multiple inheritance.

```
class className(list of superClassNames) :
```

variable **self** is used to refer to the current object.

Observe the *Car* and *Bus* classes override the superclass attribute.

```
In [140]: class Vehicle():           # this is the superclass
            wheels=0
            def __init__(self, name): # constructor is called when the object is initiated
                self.name = name + " vehicle"

            class Car(Vehicle):        # this class inherit Vehicle
                def __init__(self, name): # constructor is called when the object is initiated
                    self.name=name
                    self.wheels=4

            class Bus(Vehicle):        # this class inherit Vehicle
                def __init__(self, name): # constructor is called when the object is initiated
                    super().__init__(name) # call the superclass method
                    self.wheels=6         # override the superclass property

myCar = Car("Toyota")
print(myCar.name)           # observe there is no "vehicle" here
print(myCar.wheels)

myBus = Bus("Ford")
print(myBus.name)
print(myBus.wheels)

myVehicle=Vehicle("Proton")
print(myVehicle.name)
print(myVehicle.wheels)

Toyota
4
Ford vehicle
6
Proton vehicle
0
```

Polymorphism

Polymorphism means many forms. The function below can receive many different types of object as long as the object has *wheels* property.

```
In [141]: def CountWheels(name,myVeh):
            print(name, " has ", myVeh.wheels, " wheels")

CountWheels("BMW",myCar)
CountWheels("GM",myBus)
CountWheels("Mazda",myVehicle)

BMW has 4 wheels
GM has 6 wheels
Mazda has 0 wheels
```

Date and Time

Date and Time are object in Python. We need to import the module first

In [142]: `import datetime`

```
t = datetime.time(1, 2, 3)
print(t)
print('hour  :', t.hour)
print('minute:', t.minute)
print('second:', t.second)
print('microsecond:', t.microsecond)
print('tzinfo:', t.tzinfo)
```

```
01:02:03
hour  : 1
minute: 2
second: 3
microsecond: 0
tzinfo: None
```

In [143]: `t1 = datetime.date.today()`
`print(t1)`
`print('ctime:', t1.ctime())`
`print( 'tuple:', t1.timetuple())`
`print( 'ordinal:', t1.toordinal())`
`print( 'Year:', t1.year)`
`print( 'Mon :', t1.month)`
`print( 'Day :', t1.day)`

```
2017-09-04
ctime: Mon Sep  4 00:00:00 2017
tuple: time.struct_time(tm_year=2017, tm_mon=9, tm_mday=4, tm_hour=0, tm_min=0, tm_sec=0, tm_wday=0, tm_yday=247, tm_isdst=-1)
ordinal: 736576
Year: 2017
Mon  : 9
Day  : 4
```

In [144]: `print( "Current date and time: " , datetime.datetime.now().strftime("%c"))` *# Localized representation*
`print( "Format as you wish: " ,datetime.datetime.now().strftime("let's meet on %d/%B, %y at %H:%M:%S"))`
`print( "Current year: " , datetime.date.today().strftime("%Y"))`
`print( "Month of year: " , datetime.date.today().strftime("%m"))`
`print( "Day of the month : " , datetime.date.today().strftime("%d"))`
`print( "Day of week: " , datetime.date.today().strftime("%A"))`
`print( "Weekday of the week: " , datetime.date.today().strftime("%w"))`
`print( "Week number of the year: " , datetime.date.today().strftime("%W"))`
`print( "Day of year: " , datetime.date.today().strftime("%j"))`

```
Current date and time:  Mon Sep  4 13:03:14 2017
Format as you wish:  let's meet on 04/September, 17 at 13:03:14
Current year:  2017
Month of year:  09
Day of the month :  04
Day of week:  Monday
Weekday of the week:  1
Week number of the year:  36
Day of year:  247
```

In [145]: `d1 = datetime.date(2018, 12, 31)`
`print('d1:', d1.strftime("%a"))`

```
d1: Mon
```

```
In [146]: d2=datetime.datetime(2019, 12, 31, 23, 55, 25)
print(d2.strftime("%A %d %b %Y %H:%M:%S"))
```

Tuesday 31 Dec 2019 23:55:25

Use *Try* and *except Exception as err* to deal with error.

```
In [147]: def dayInWeek(row):
          try:
              d1 = datetime.date(row['Year'], row['Month'], row['Date'])
              return d1.strftime("%a")
          except Exception as err:
              return math.nan

data=[{'Year': 2016, 'Month': 2, 'Date': 29},
       {'Year': 2018, 'Month': 12, 'Date': 30},
       {'Year': 2019, 'Month': 0, 'Date': 30},
       {'Year': 2017, 'Month': 9, 'Date': 2}]

list(map(lambda row: dayInWeek(row), data))
```

```
Out[147]: ['Mon', 'Sun', nan, 'Sat']
```